

Računarska vežba 10

PYTHON ALGORITMI SORTIRANJA

1 Algoritam Merge Sort

Prosečna vremenska složenost algoritama od $O(n^2)$ ne čini ih prikladnim za sortiranje velikih nizova, pa se tada mogu koristiti brži algoritmi koji koriste strategiju "podeli pa vladaj".

Njihova prosečna vremenska složenost je $O(n \log_2 n)$, a Merge Sort je najpoznatiji među njima. Radi na način da se ulazni niz rekurzivno deli na dva dela sve dok ne se dođe do jednog elementa. Zatim se generisani podnizovi spajaju nazad na sortirani način korišćenjem funkcije MERGE (u prikazanom pseudokodu)

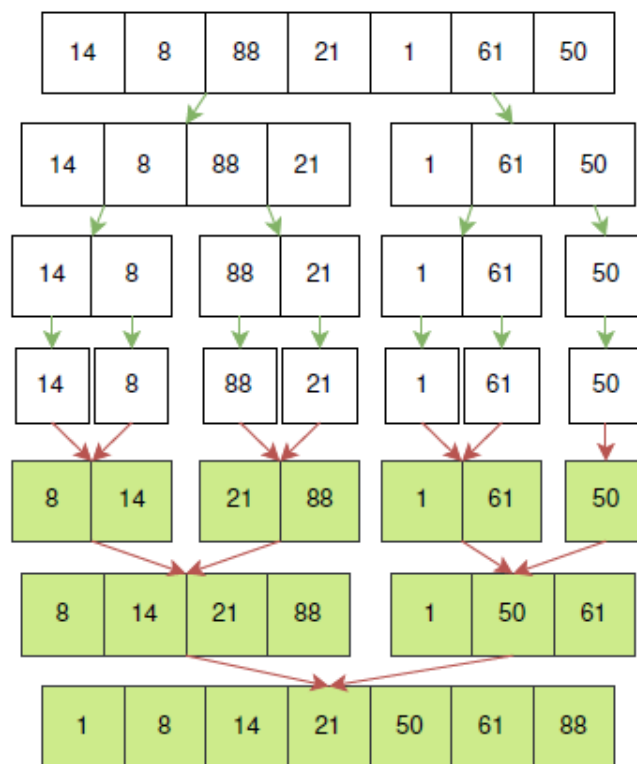
Pseudokod MergeSort

```
function MERGE(levo, desno)
    rezultat ← []
    while levo.veličina-niza > 0 and desno.veličina-niza > 0 do
        if levo[1] ≤ desno[1] then
            dodaj levo[1] u rezultat
            ukloni levo[1] iz levo
        else
            dodaj desno[1] u rezultat
            ukloni desno[1] iz desno
        end if
    end while
    dodaj levo u rezultat
    dodaj desno u rezultat
return rezultat
end function
```

Primer za Merge Sort algoritam:

Pretpostavimo da imamo niz brojeva [14, 8, 88, 21, 1, 61, 50] koje treba sortirati. Postupak sortiranja započinje deljenjem postojećeg niza u dva dela. Nakon deljenja izvornog niza dobili smo dva podniza: [14, 8, 88, 21] i [1, 61, 50]. Na isti način dele se levi i desni podniz. Tada dobijamo 4 podniza: [14, 8], [88, 21], [1, 61] i [50], i u zadnjem deljenju 7 podnizova: [14], [8], [88], [21], [1], [61] i [50].

Podnizove više nije moguće dalje deliti i počinje njihovo spajanje. U tom procesu podnizovi se spajaju na način da su njihovi elementi sortirani. Tako se generišu sledeći podnizovi: [8, 14], [21, 88], [1, 61] i [50]. Njihovim spajanjem dobijamo dva veća sortirana podniza: [8, 14, 21, 88] i [1, 50, 61] i u zadnjem spajanju konačni sortirani niz.



Programski kôd:

Funkcija `merge_sort` se rekurzivno poziva na levoj i desnoj polovini ulaznog niza dok se ne dostigne osnovni slučaj (kada niz ima samo jedan element).

- Niz se deli na dve polovine pronalaženjem srednjeg indeksa (`mid`), i kreiraju se levi i desni podniz
- Funkcija `merge_sorted_arrays` vrši spajanje sortiranih levih i desnih nizova u jedan sortirani niz
- Proces spajanja se odvija upoređivanjem elemenata iz oba niza (`left_arr` i `right_arr`) i dodavanjem manjeg elementa u `sorted_arr`
- Nakon iscrpljivanja jednog od nizova, svi preostali elementi u drugom nizu se dodaju u `sorted_arr`
- Na kraju, funkcija `merge_sorted_arrays` vraća sortirani niz

```

1 def merge_sort(array):
2     if len(array) <= 1:
3         return array
4     mid = len(array) // 2
5     left_arr = array[:mid]
6     right_arr = array[mid:]
7
8     return merge_sorted_arrays(merge_sort(left_arr), merge_sort(right_arr))
9
10 def merge_sorted_arrays(left_arr, right_arr):
11     sorted_arr = []
12     i = j = 0
13     while i < len(left_arr) and j < len(right_arr):
14         if left_arr[i] <= right_arr[j]:
15             sorted_arr.append(left_arr[i])
16             i += 1
17         else:
18             sorted_arr.append(right_arr[j])
19             j += 1
20
21     sorted_arr.extend(left_arr[i:])
22     sorted_arr.extend(right_arr[j:])
23
24     return sorted_arr
25
26 arr = [123, 87, 66, 49, 37, 25, 13, 1]
27 print(f"Ulazni niz: arr = {arr}")
28 print("Sortirani niz:", merge_sort(arr))
--

```

Ulazni niz: arr = [123, 87, 66, 49, 37, 25, 13, 1]
 Sortirani niz: [1, 13, 25, 37, 49, 66, 87, 123]

Primer za vežbu:

Korišćenjem prethodnog primera napisati programski kod za Merge Sort sortiranje niza, koji će kao izlaz imati npr. sledeće opcije.

```

Unesi donju granicu opsega: 100
Unesi gornju granicu opsega: 500
Unesi broj elemenata liste za sortiranje: 15
Originalna lista: [277, 225, 153, 377, 166, 110, 239, 310, 124, 334, 396, 461, 144, 229, 199]
Sortirana lista: [110, 124, 144, 153, 166, 199, 225, 229, 239, 277, 310, 334, 377, 396, 461]

```

Potrebno je uneti broj elemenata liste (niza) koji se sortira, donju i gornju granicu opsega niza i korišćenjem metode `random.randint()` omogućiti unos slučajnih brojeva u zadatom opsegu sortiranja.

```

1  import random
2
3  def merge(left, right):
4      result = []
5      i = j = 0
6      while i < len(left) and j < len(right):
7          if left[i] <= right[j]:
8              result.append(left[i])
9              i += 1
10         else:
11             result.append(right[j])
12             j += 1
13     result.extend(left[i:])
14     result.extend(right[j:])
15     return result
16
17 def merge_sort(arr):
18     if len(arr) <= 1:
19         return arr
20     mid = len(arr) // 2
21     left = merge_sort(arr[:mid])
22     right = merge_sort(arr[mid:])
23     return merge(left, right)
24
25 # Unos opsega i elemenata liste
26 low = int(input("Unesi donju granicu opsega: "))
27 high = int(input("Unesi gornju granicu opsega: "))
28 size = int(input("Unesi broj elemenata liste za sortiranje: "))
29
30 # Generisanje random celih brojeva
31 nums = [random.randint(low, high) for _ in range(size)]
32 print("Originalna lista:", nums)
33
34 # Sortiranje
35 sorted_nums = merge_sort(nums)
36 print("Sortirana lista:", sorted_nums)

```

```

Unesi donju granicu opsega: 100
Unesi gornju granicu opsega: 500
Unesi broj elemenata liste za sortiranje: 15
Originalna lista: [277, 225, 153, 377, 166, 110, 239, 310, 124, 334, 396, 461, 144, 229, 199]
Sortirana lista: [110, 124, 144, 153, 166, 199, 225, 229, 239, 277, 310, 334, 377, 396, 461]

```

2 Algoritam Quick Sort

Quick Sort je jedan od najbržih algoritama za sortiranje. Koristi strategiju "podeli pa vladaj" i ima vremensku složenost **O(nlogn)**. Osnovna ideja je da se u odnosu na odabrani referentni element (pivot) niz sortira tako da svi brojevi manji od njega budu sa leve strane niza, a svi veći sa desne strane. Zatim, u zavisnosti od položaja pivot elementa, niz se rekursivno deli na dva dela, tj. deo pre i deo posle pivot elementa, nakon čega se prethodno opisani postupak ponavlja za oba nova podniza.

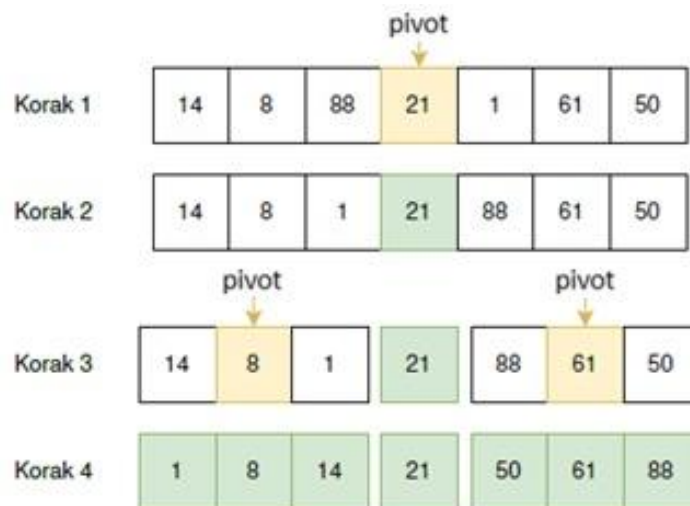
Quick Sort postoji u mnogo varijanti, koje se razlikuju u načinu izbora pivot elementa, koji može biti odabran kao prvi, zadnji, srednji ili slučajni element niza. Prikazan je pseudokod implementacije algoritma Quick Sort kada je pivot element srednji element niza.

```
function QUICK-SORT(niz, donji, gornji)
  if donji < gornji then
    pivot ← (donji + gornji)/2 ▷ lokacija pivot elementa
    pivot ← PARTITION(niz, donji, gornji, pivot)
    QUICK-SORT(niz, donji, pivot - 1)
    QUICK-SORT(niz, pivot + 1, gornji)
  end if
end function

function PARTITION(niz, donji, gornji, pivot)
  zameni niz[pivot] i niz[gornji] ▷ pomeri pivot element na kraj
  pivot ← niz[gornji]
  i ← donji - 1
  for j ← donji .. gornji - 1 do
    if niz[j] ≤ pivot then
      i ← i + 1
      zameni niz[i] i niz[j]
    end if
  end for
  zameni niz[i + 1] i niz[gornji] ▷ postavi pivot element na tačnu poziciju
  return i + 1
end function
```

Primer za Quick Sort algoritam:

Pretpostavimo da imamo niz niz brojeva: **14, 8, 88, 21, 1, 61, 50**



U početnom nizu od sedam elemenata biramo srednji element (broj 21) kao pivot element niza. Zatim, u drugom koraku sve brojeve manje od 21 prebacujemo levo, tj. pre pivot elementa, a sve veće na desnu

stranu. Kao rezultat dobijamo niz gde je broj 21 na ispravnom (sortiranom) mestu pa možemo pristupiti podeli inicijalnog niza u 2 dela. U trećem koraku sada postoje dva podniza – jedan sa elementima levo od pivot elementa i jedan sa elementima desno od pivot elementa. Na svakom od tih podnizova primenjujemo isti postupak, odnosno biramo srednji element niza kao pivot element te manje vrednosti smestamo levo, a veće desno od pivot elementa. U četvrtom koraku celi niz je sortiran.

Programski kôd :

```

1  def quick_sort(array):
2      quick_sort_helper(array, 0, len(array) - 1)
3      return array
4
5  def quick_sort_helper(array, start, end):
6      # Osnovni slučaj: podniz ima 0 ili 1 element
7      if start >= end:
8          return
9
10     # Izbor prvog elementa kao pivot
11     pivot = start
12     left = start + 1
13     right = end
14
15     while right >= left:
16         # Zamena elemenata ako su sa pogrešne strane pivota...
17         if array[left] > array[pivot] and array[right] < array[pivot]:
18             array[left], array[right] = array[right], array[left]
19
20         if array[left] <= array[pivot]:
21             left += 1
22
23         if array[right] >= array[pivot]:
24             right -= 1
25
26     # Premestiti pivot na njegovu sortiranu poziciju
27     array[pivot], array[right] = array[right], array[pivot]
28
29     # Rekurzivno sortiranje podnizova
30     is_left_subarray_smaller = right - 1 - start < end - (right + 1)
31
32     if is_left_subarray_smaller:
33         quick_sort_helper(array, start, right - 1)
34         quick_sort_helper(array, right + 1, end)
35     else:
36         quick_sort_helper(array, right + 1, end)
37         quick_sort_helper(array, start, right - 1)
38
39 arr = [134, 95, 74, 55, 32, 11, 8, 2]
40 print(f"Ulazni niz: arr = {arr}")
41 print("Sortirani niz:", quick_sort(arr)) # Output [1, 3, 5, 7, 9]
42

```

- Ulazni niz: arr = [134, 95, 74, 55, 32, 11, 8, 2]
Sortirani niz: [2, 8, 11, 32, 55, 74, 95, 134]

3 Algoritam Radix Sort

Uz neke prepostavke o podacima i upotrebom podataka na specifičan način može se postići linearna složenost algoritma sortiranja.

Jedno od rešenja je „sortiranje višestrukim razvrstavanjem“ ili Radix Sort. Sortiranje višestrukim razvrstavanjem predstavlja prirodno uopštenje najjednostavnijeg postupka sortiranja koji se sastoji u osiguravanju dovoljnog broja lokacija kako bi se svaki element smestio na svoju lokaciju. Radix sortiranje koristi pozicionu reprezentaciju broja (ili niza karaktera) i odvojeno analizira cifre (ili znakove) na raznim pozicijama.

Ulazni niz ili lista sadrži n ulaznih elemenata koji se sastoje od celih brojeva (digits) sa k cifara $d_{k-1}, d_{k-2}, \dots, d_0$

U prvom koraku vrši se sortiranje po najmlađoj cifri (cifra jedinica). Uzimaju se redom elementi iz neuređenog niza i razvrstavaju u deset redova $Q_0, \dots, Q_9$ po vrednosti najmlađe cifre, tako što se tekući element stavlja na kraj odgovarajućeg reda. Kad se obrade svi elementi, svi redovi se spoje u jedan niz po poretku $Q_0, Q_1 \dots Q_9$

U sledećem koraku se opet uzimaju elementi ovog niza redom i razvrstavaju u redove, ali po vrednosti druge cifre (desetice) d_1 , pa se opet na kraju svi redovi spoje.

Postupak se završava kad se izvrši uređenje i po najstarijoj cifri d_{k-1} .

Pošto se u svakom od k koraka obrađuju svih n elemenata, vremenska složenost ovog algoritma je $O(kn)$

- **PRIMER:** Niz veličine $k=6, d=5$

A	33101	26440	16341	2332	20101	801
----------	-------	-------	-------	------	-------	-----

- ▶ Raspored cifara jedinica: 1 0 1 2 1 1 → Sortiranje prebrojavanjem: 0 1 1 1 1 2

A	26440	33101	16341	20101	801	2332
----------	-------	-------	-------	-------	-----	------

- ▶ Raspored cifara desetica: 4 0 4 0 0 3 → Sortiranje prebrojavanjem: 0 0 0 3 4 4

A	33101	20101	801	2332	26440	16341
----------	-------	-------	-----	------	-------	-------

- ▶ Raspored cifara stotina: 1 1 8 3 4 3 → Sortiranje prebrojavanjem: 1 1 3 3 4 8

A	33101	20101	2332	16341	26440	801
----------	--------------	--------------	-------------	--------------	--------------	------------

- Raspored cifara hiljada: 3 0 2 6 6 0 → Sortiranje prebrojavanjem: 0 0 2 3 6 6

A	20101	801	2332	33101	16341	26440
----------	--------------	------------	-------------	--------------	--------------	--------------

- Raspored cifara deset hiljada: 2 0 0 3 1 2 → Sortiranje prebrojavanjem: 0 0 1 2 2 3
 ► **Konačno sortiran niz:**

A	801	2332	16341	20101	26440	33101
----------	------------	-------------	--------------	--------------	--------------	--------------

Programski kôd:

```

1  def radix_sort(array):
2      if len(array) == 0:
3          return array
4      max_number = max(array)
5      num_digits = len(str(max_number))
6      for digit in range(num_digits):
7          counting_sort(array, digit)
8      return array
9
10 def counting_sort(array, digit):
11     sorted_array = [0] * len(array)
12     count_array = [0] * 10
13
14     for num in array:
15         count_index = (num // (10 ** digit)) % 10
16         count_array[count_index] += 1
17
18     for idx in range(1, 10):
19         count_array[idx] += count_array[idx - 1]
20
21     for idx in range(len(array) - 1, -1, -1):
22         count_index = (array[idx] // (10 ** digit)) % 10
23         count_array[count_index] -= 1
24         sorted_index = count_array[count_index]
25         sorted_array[sorted_index] = array[idx]
26
27     for idx in range(len(array)):
28         array[idx] = sorted_array[idx]
29
30 array = [321, 56, 789, 12, 456, 234, 901]
31 print(f"Ulazni niz: {array}")
32 radix_sort(array)
33 print("Sortirani niz:", array)

```

- Ulazni niz: [321, 56, 789, 12, 456, 234, 901]
 Sortirani niz: [12, 56, 234, 321, 456, 789, 901]